Homework 6
Computer System Organization
ECE 361
Fall 2026

Overview

Homework 6 builds the part of your project that is the same for every device: reading the sensor and the switches through iom361's registers, lighting LEDs without disturbing the others, keeping the clock, and replaying a scenario file into a trace, one line per tick. Here the device is simple, **a press of switch k toggles LED k**; in your project, your state machine takes its place. Due **Sun, Nov 8, 11:59pm**, pushed to `hw06/`. AI tools are allowed. Your Shine about your code is due by **Thu, Nov 12, 11:59pm** (Wednesday is Veterans Day), without any supporting materials. Satisfactory / Not Yet, as before.

Part 1: The sensor

In `hw06/sensor.h`, the `reading_t` from your project sheet (`time_t t`, `float temp_c`, `float rh_pct`) and

- `int sensor_read(uint32_t *base, time_t now, reading_t *r)`: reads `TEMP_REG` and `HUMID_REG`, **keeps bits 19..0** of each, converts them with the formulas in `iom361_r4.h`, **rounds to 0.1**, and stores them with the timestamp `now` in `*r`. Returns 0, or the iom361 return code of a failed read; then it changes nothing.
- No `printf` in `sensor.c`, and no calls to the test functions `_iom361_set_…`: on a real board they would not exist.

Part 2: Switches and LEDs

In `hw06/panel.h` and `panel.c`:

Table 1. The panel functions.

Function	What it does
`uint32_t panel_pressed(uint32_t now, uint32_t before)`	the switches that went from off to on, one bit per switch
`uint32_t panel_released(uint32_t now, uint32_t before)`	the switches that went from on to off
`int panel_led(uint32_t *base, int n, int on)`	turns LED n on or off **by read-modify-write**, so that every other LED keeps its state; −1 if n is not 0 to 31
`int panel_toggle(uint32_t *base, int n)`	toggles LED n, the same way

Part 3: The scenario and the loop

- `hw06/scenario.h` and `scenario.c`: load a scenario file in the format of your project sheet (comments with #, a `start <timestamp>` line, then `tick temp rh switches` with the switches in hexadecimal and increasing ticks) into a `scenario_t` that holds at most 1000 lines, **no malloc**. `scenario_load` returns 0, or the number of the first wrong line. A function gives the line in force at a tick: the values hold between lines.
- `hw06/main.c`: `./replay scenario.txt`. It initializes iom361 with 4 switches and 4 LEDs and turns the display off. For every tick from 0 to the last one: the line in force sets the "world" with `_iom361_set_sensor1` and `_iom361_set_switches`; the clock is `start + tick`; it reads the sensor and the switches **through the registers**, toggles LED k for every switch k just pressed, and prints one line:

```
2026-10-06 08:00:02 temp=21.5 rh=40.0 switches=0x1 pressed=0x1 leds=0x1
```

The time is printed in UTC with `gmtime` and `strftime`, so a trace is the same on every computer.

Part 4: Tests and a Makefile

- `hw06/tests/test_hw06.c` tests, at least: a reading of 91.0 °C reads back as 91.0; a failed read changes nothing; pressed and released for a press, a release, and a held switch; `panel_led` and `panel_toggle` keep the other LEDs; a load with comments and blank lines; values holding between lines; and a scenario without a start line, with a missing field, and with ticks that do not increase. PASS or FAIL per test, a nonzero exit status on failure.
- **A regression test:** at least two scenarios in `hw06/scenarios/` with their expected traces in `hw06/expected/`, and `make test` runs the unit tests, then compares `./replay` on each scenario with its expected trace (a `regress.sh` with `diff`, as in HW2).
- `hw06/Makefile`: `make` builds `replay` from all modules and `iom361_r4.c` with no warnings; each `.o` lists the headers it includes; `make test`; `make clean`.

Your project: a next step (not graded)

Scaffolding, not graded and not on the Shine: copy `sensor.c`, the scenario code, and the loop into `project/`, replace the toggling with your state machine from HW5's project step, and print your device's trace format from your sheet. Your first scenarios from the sheet now run.

Deliverable, Shine, and requirements

- **Push by Sun, Nov 8, 11:59pm, in hw06/:** `sensor.h`, `sensor.c`, `panel.h`, `panel.c`, `scenario.h`, `scenario.c`, `main.c`, `iom361_r4.h`, `iom361_r4.c`, `tests/test_hw06.c`, `scenarios/`, `expected/`, `regress.sh`, `Makefile`, `README.md` (how to build and test), `AI_USAGE.md`.
- **Shine:** opens Mon, Nov 9, in the morning; due Thu, Nov 12, 11:59pm; 10 to 15 minutes about your own code, with questions like "Why does your panel_led read the register before it writes it, and what goes wrong if it doesn't?"